Relational Database Concepts

Tables, relationships, normalization (1NF–3NF); when to use RDBMS

Rahul Guha || SME || Java FS Training || HCL

5. SQL & Database

Relational Database Concepts

What is a Database?

A database is a structured collection of data that is organized and stored in a way that facilitates efficient retrieval, manipulation, and management. Think of it as a digital filing cabinet, where instead of physical folders and documents, you have tables and records neatly organized for easy access.

What is a Relational Database?

A relational database is a type of database that organizes data into tables (relations) with rows (records) and columns (fields). These tables are interconnected through relationships, allowing data to be accessed and combined in various ways. Imagine a collection of spreadsheets, each representing a different aspect of your data, but with the ability to link and combine information across them seamlessly.

RDBMS

A Relational Database Management System (RDBMS) is a software application designed to create, manage, and interact with relational databases. It provides a structured framework for storing, retrieving, and manipulating data within the database. Some popular examples of RDBMS include MySQL, PostgreSQL, Oracle, and Microsoft SQL Server.

Different Types of Keys in RDBMS

In a Relational Database Management System (RDBMS), a key is an attribute or a combination of attributes used to identify records uniquely, establish relationships between tables, and maintain data integrity.

1. Types of keys in RDBMS

1. Primary Key (PK)

Uniquely identifies each row in a table. It cannot contain duplicate or NULL values.

Example: StudentID

SQL:

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

2. Foreign Key (FK)

References a primary key or another eligible unique key in a related table. It maintains referential integrity.

Example: DepartmentID in Students references Departments.

SQL:

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID)
);
```

Different Types of Keys in RDBMS

3. Candidate Key

A minimal attribute or attribute combination capable of uniquely identifying a row.
A table may have multiple candidate keys.
Example: StudentID and Email, if both are individually unique and non-null.

SQL:

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE NOT NULL
);
```

4. Composite Key

A key consisting of two or more columns that together uniquely identify a row.
Example: A student-course enrollment.

SQL:

```
CREATE TABLE Enrollments (
    StudentID INT,
    CourseID INT,
    Grade VARCHAR(2),
    PRIMARY KEY (StudentID, CourseID)
);
```

A student can enroll in many courses, but the same student-course pair cannot be repeated.

5. Unique Key

A constraint that prevents duplicate values in a column or combination of columns.

SQL:

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

Different Types of Keys in RDBMS

In MySQL, a UNIQUE column can generally allow NULL values, unlike a primary key. Exact behavior depends on the database system.

6. Super Key

Any set of attributes that uniquely identifies a row, even if it contains unnecessary attributes.

Example: If StudentID is unique, both {StudentID} and {StudentID, Name} are super keys. Only {StudentID} is minimal, so it is a candidate key.

7. Alternate Key

A candidate key that was not selected as the primary key.

Example: If StudentID is the primary key and Email is another candidate key, Email is an alternate key.

8. Natural Key

A key based on a meaningful, real-world attribute. Examples: ISBN for a book or a government-issued identifier where appropriate.

SQL:

```
CREATE TABLE Books (
    ISBN VARCHAR(20) PRIMARY KEY,
    Title VARCHAR(100)
);
```

A natural key should be stable and unique in the relevant domain.

9. Surrogate Key

An artificially generated identifier with no business meaning, commonly an integer or UUID.

```
CREATE TABLE Products (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
    ProductName VARCHAR(100)
);
```

ProductID is a surrogate key. MySQL generates its value automatically.

Comparison Table

Key Type	Unique?	NULL Allowed?	Main Purpose
Primary Key	Yes	No	Identifies each row uniquely
Foreign Key	Not necessarily	Yes, unless restricted	Connects tables
Candidate Key	Yes	No under candidate-key semantics	Potential primary key
Composite Key	Yes, as a combination	Depends on constraints; PK components cannot be NULL	Uses multiple columns
Unique Key	Yes	DBMS-dependent	Prevents duplicate values
Super Key	Yes	Depends on attributes and constraints	Uniquely identifies rows
Alternate Key	Yes	No under candidate-key semantics	Candidate key not selected as primary key
Natural Key	Yes	Depends on implementation	Uses meaningful business data
Surrogate Key	Yes	No when used as a primary key	Provides an artificial identifier

SQL code: Create and populate the Employee table

```
CREATE TABLE IF NOT EXISTS Employee (
   Empno INTEGER PRIMARY KEY,
   Ename TEXT NOT NULL,
   Mail_id TEXT UNIQUE,
   Aadhaar_No TEXT UNIQUE,
   Driving_license TEXT UNIQUE
);

INSERT OR IGNORE INTO Employee
   (Empno, Ename, Mail_id, Aadhaar_No, Driving_license)
VALUES
   (101, 'Ravi',    'ravi@example.com',    'DUMMY-AAD-0001', 'DL-DUMMY-0001'),
   (102, 'John',    'john@example.com',    'DUMMY-AAD-0002', 'DL-DUMMY-0002'),
   (103, 'Ravi',    'ravi1@example.com',   'DUMMY-AAD-0003', 'DL-DUMMY-0003'),
   (104, 'Krishna', 'krishna@example.com', 'DUMMY-AAD-0004', 'DL-DUMMY-0004'),
   (105, 'John',    'john12@example.com',  'DUMMY-AAD-0005', 'DL-DUMMY-0005');
```

SQL code: Create and populate the Employee table

	Empno	Ename	Mail_id	Aadhaar_No	Driving_license
	Filter	Filter	Filter	Filter	Filter
1	101	Ravi	ravi@example.com	DUMMY-AAD-0001	DL-DUMMY-0001
2	102	John	john@example.com	DUMMY-AAD-0002	DL-DUMMY-0002
3	103	Ravi	ravi1@example.com	DUMMY-AAD-0003	DL-DUMMY-0003
4	104	Krishna	krishna@example.com	DUMMY-AAD-0004	DL-DUMMY-0004
5	105	John	john12@example.com	DUMMY-AAD-0005	DL-DUMMY-0005

Identify the keys in your Employee table

SQL code: Create and populate the Employee table

Column	Key type	Reason
`Empno`	**Primary Key (PK)**	Uniquely identifies each employee; cannot be `NULL` or duplicated.
`Ename`	Not a key	Employee names can repeat. Ravi and John each appear twice.
`Mail_id`	**Candidate Key and Alternate Key**	It is unique and can identify an employee. Because `Empno` is already the primary key, `Mail_id` is an alternate key.
`Aadhaar_No`	**Unique key constraint**	Prevents duplicate non-`NULL` values. It could be a candidate key if it is also declared `NOT NULL`.
`Driving_license`	**Unique key constraint**	Prevents duplicate non-`NULL` values. It could be a candidate key if it is also declared `NOT NULL`.

Identify the keys in your Employee table

Now create another Table

```
//Create the Department table
CREATE TABLE IF NOT EXISTS Department (
    Dept_No INTEGER PRIMARY KEY,
    DName TEXT NOT NULL UNIQUE,
    Location TEXT NOT NULL
);
//Insert the dummy data
INSERT OR IGNORE INTO Department
    (Dept_No, DName, Location)
VALUES
    (10, 'Accounts',   'Hyderabad'),
    (20, 'Sales',      'Delhi'),
    (30, 'Research',   'Mumbai'),
    (40, 'Operations', 'Chennai');
```

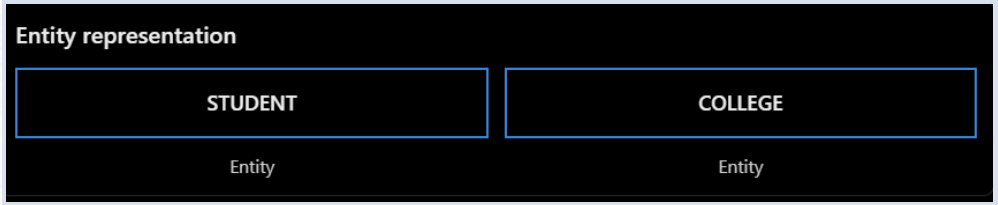
Dept_No	DName	Location
10	Accounts	Hyderabad
20	Sales	Delhi
30	Research	Mumbai
40	Operations	Chennai

Empno	Ename	Mail_id	Aadhaar_No	Driving_license	Dept_No
101	Ravi	ravi@example.com	DUMMY-AAD-0001	DL-DUMMY-0001	10
102	John	john@example.com	DUMMY-AAD-0002	DL-DUMMY-0002	20
103	Ravi	ravi1@example.com	DUMMY-AAD-0003	DL-DUMMY-0003	30
104	Krishna	krishna@example.com	DUMMY-AAD-0004	DL-DUMMY-0004	40
105	John	john12@example.com	DUMMY-AAD-0005	DL-DUMMY-0005	20

Entity–Relationship (ER) Model: Entity and Weak Entity

What is an Entity?

An entity is a real-world object, person, place, or concept about which data is stored in a database.
Examples: Student, College, Employee, Department, Bank Account, Course.
An entity is represented by a rectangle in an ER diagram.



•Example: Student entity

Here, each student is an entity instance, and StudentID is the primary key that uniquely identifies each student.

StudentID (PK)	StudentName	Email
101	Aman	aman@example.com
102	Priya	priya@example.com
103	Rahul	rahul@example.com

Characteristics of an entity

-> Represents a real-world object or concept.
-> Has attributes that describe its properties.
-> Usually has a key that uniquely identifies its instances.
-> Is represented by a rectangle in an ER diagram.
-> Can participate in relationships with other entities.

Types of entities

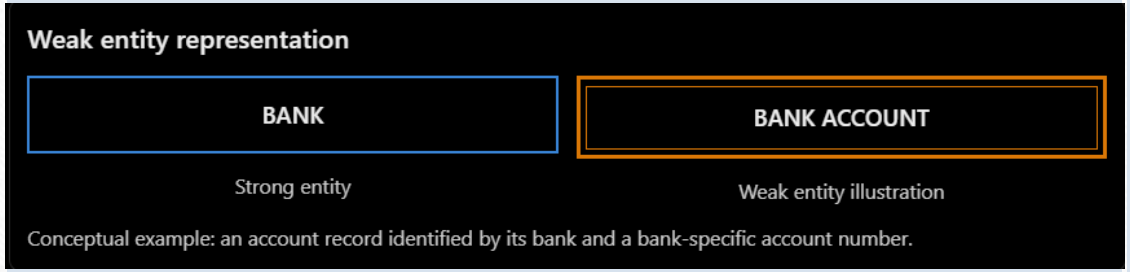
1. Strong entity: Can be uniquely identified using its own key.
2. Weak entity: Cannot be uniquely identified using its own attributes alone; it depends on an owner entity.

2. What is a Weak Entity?

A weak entity is an entity that does not have a complete key of its own to uniquely identify each instance.

Entity–Relationship (ER) Model: Entity and Weak Entity

It depends on an associated strong entity for identification.
A weak entity is represented by a double rectangle in a traditional Chen ER diagram.



Example: Bank and Bank Account
Suppose two banks both have an account numbered 1001.

BankID	BankName	AccountNo	Balance
B01	City Bank	1001	25,000
B02	National Bank	1001	40,000

AccountNo alone is not globally unique. The account can be identified using the combination of BankID and AccountNo.
- Strong entity: Bank
- Weak entity: BankAccount
in this example
- Owner key: BankID
- Partial key: AccountNo
- Combined identifier: (BankID, AccountNo)

Important:
A bank account is not inherently a weak entity in every database design.
It is weak in this example because its identity depends on the owning bank.
A weak entity's partial key distinguishes its instances under the same owner.

Entity–Relationship (ER) Model

1. Definition of ER Model

An Entity–Relationship (ER) Model describes the logical structure of a database using entities, attributes, and relationships. Its graphical representation is called an ER Diagram (ERD).

- An ER model acts as a blueprint for database design.

- It helps identify data objects, their properties, and their relationships.

- It can later be translated into relational tables.

- The two fundamental concepts are entity sets and relationship sets.

2. Main components of an ER ModelThe following diagram follows the structure of your reference image.

There are three main branches:

- **Entity — the object or concept being represented.**
- **Attribute — a property of an entity or relationship.**
- **Relationship — an association between entities.**

3. Entity

An entity is a real-world object or concept about which data is stored.

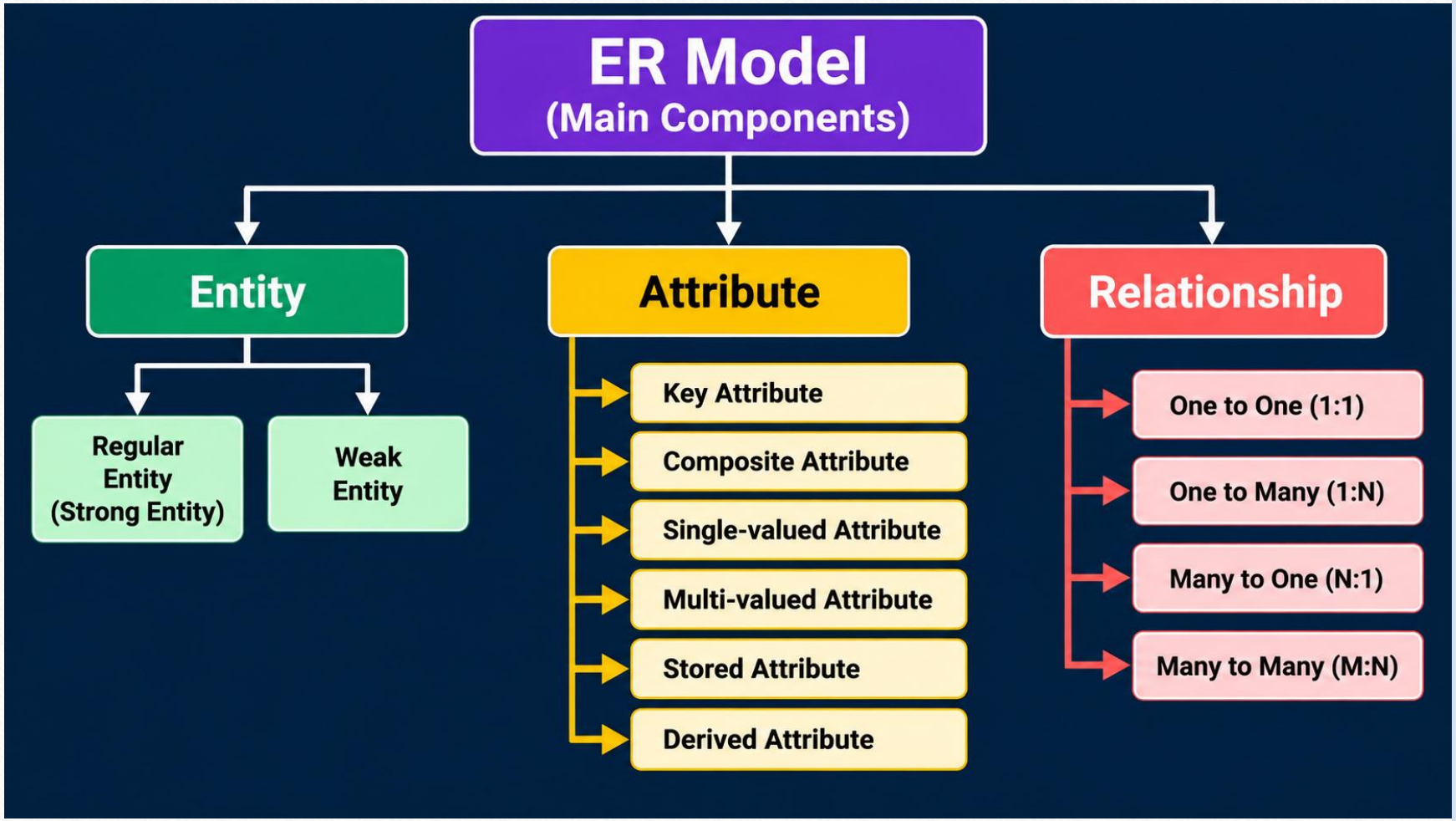
Examples: Student, College, Employee, Department.

- Representation: Rectangle.

- Entity instance: One specific object, such as student Rahul.

- Entity set: Collection of similar entities, such as all students.

Entity–Relationship (ER) Model



5. Relationships

A relationship represents an association between two or more entities.

Examples:

- Student enrolls in Course.
- Department employs Employee.
- College admits Student.

In a traditional Chen ER diagram, a relationship is represented by a diamond.

Entity–Relationship (ER) Model

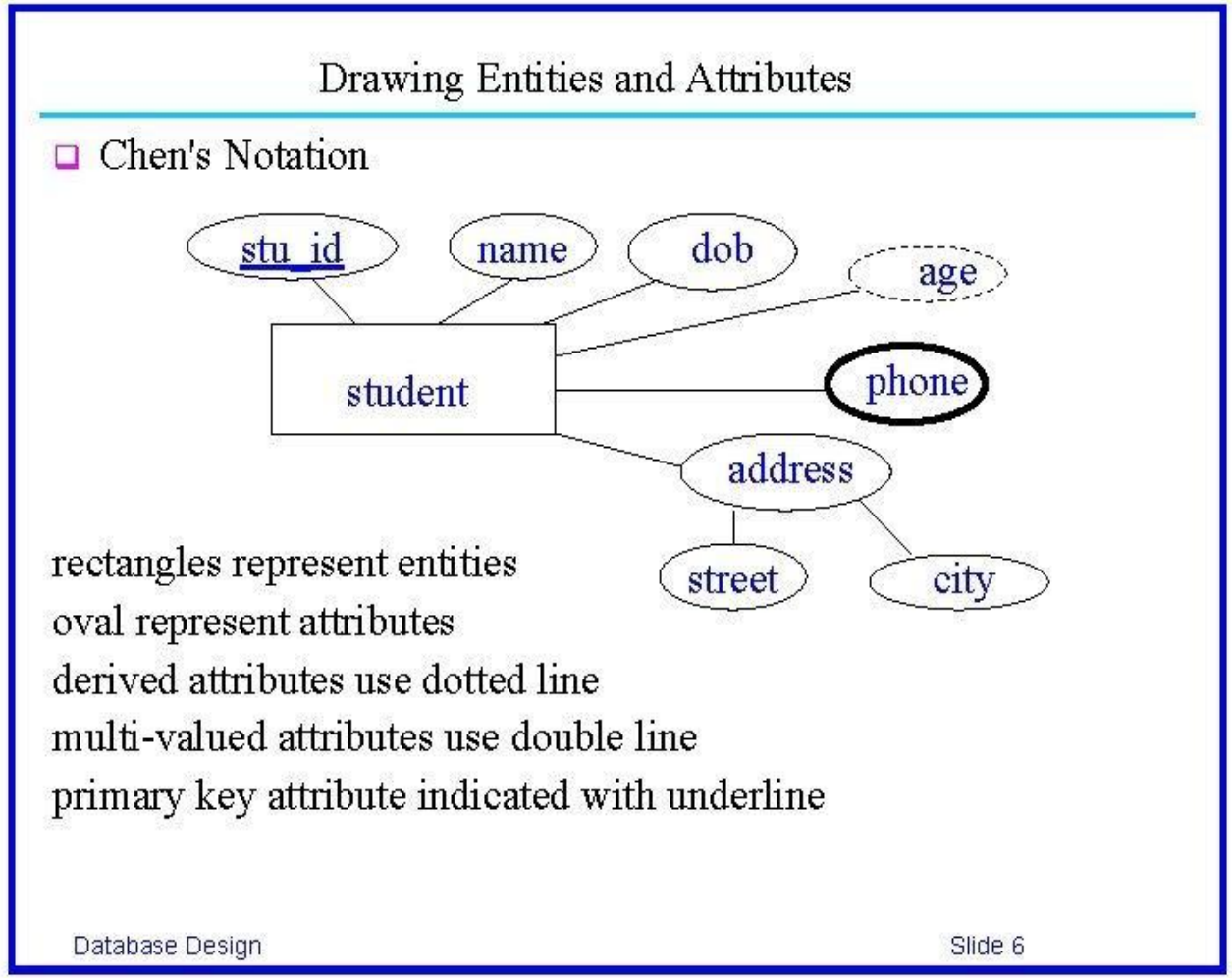
1. Key Attribute : Uniquely identifies an entity instance. In a Chen ER diagram, its name is underlined.
Example: StudentID.

2. Composite Attribute : Can be divided into smaller meaningful attributes.
Example: Name → FirstName, MiddleName, LastName.

3. Simple Attribute: An Attribute that can not be divided into simpler component.
Example: RollNo, Age

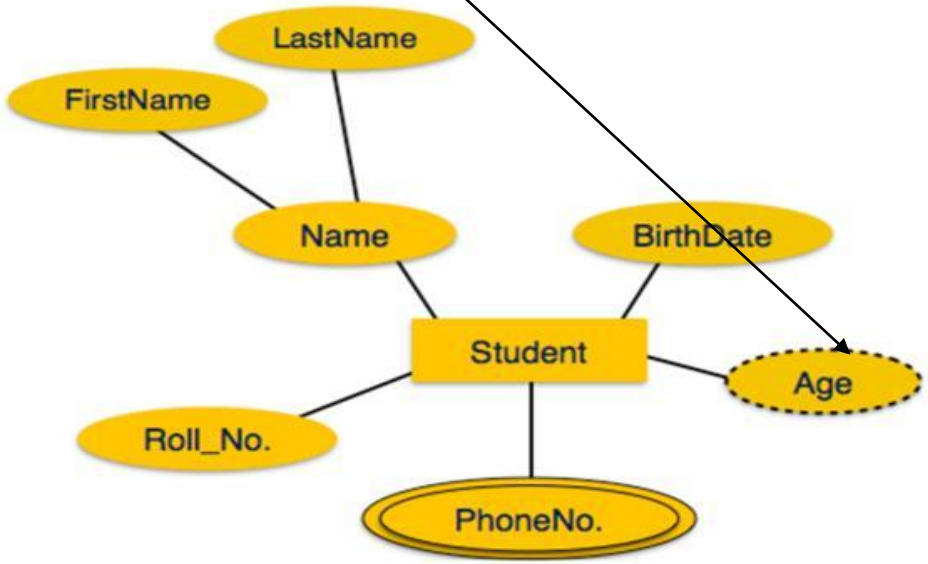
4. Single-Valued Attribute: Has one value for each entity instance at a given time.
Example: DateOfBirth.

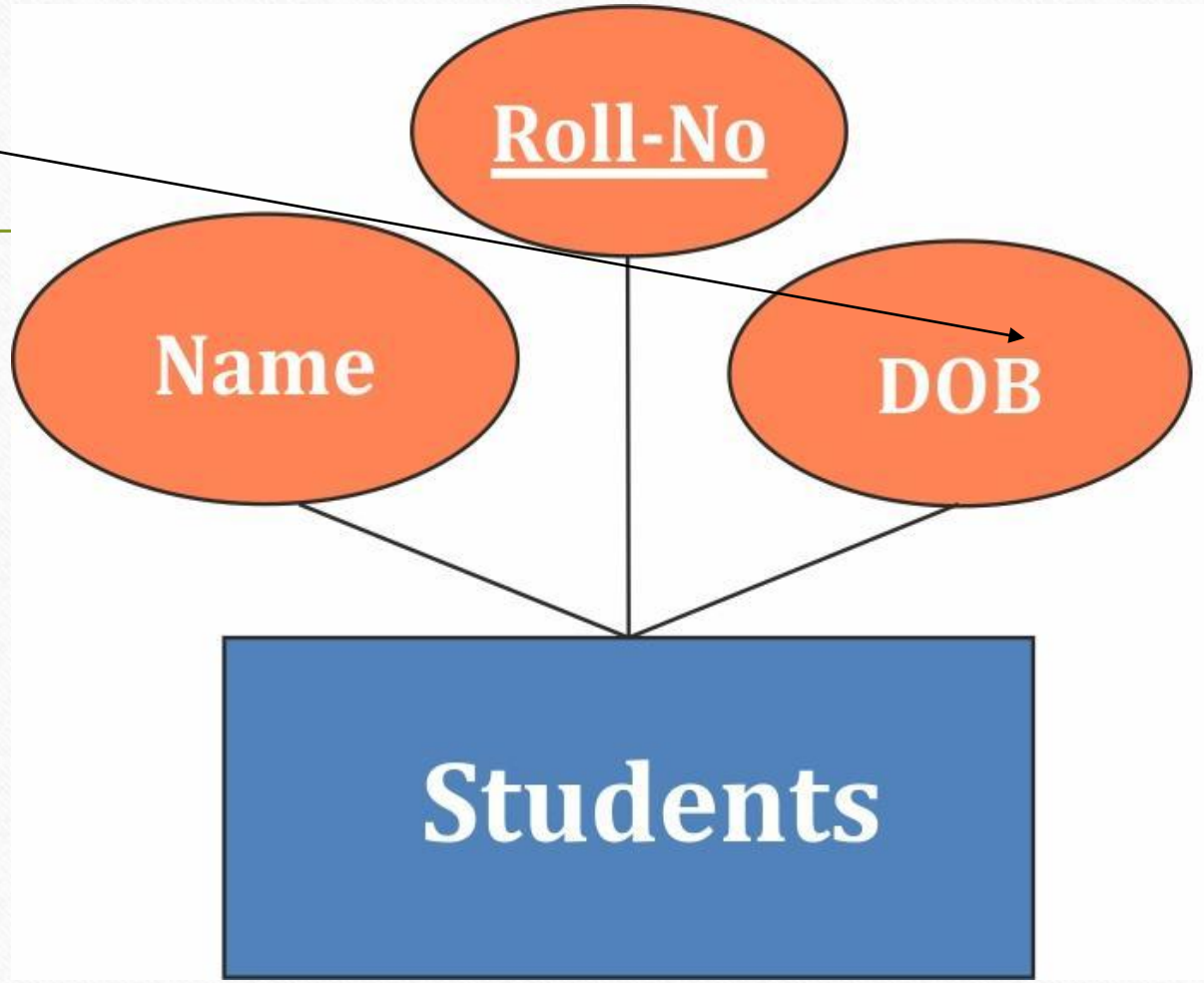
4. Multi-Valued Attribute
Can have multiple values for one entity instance. Represented by a double oval.
Example: Phone Numbers.



Entity–Relationship (ER) Model

5. Stored Attribute
Its value is stored in the database.
Example: DateOfBirth.
6. Derived Attribute
Its value is calculated from another attribute.
Represented by a dashed oval.
Example: Age, calculated from DateOfBirth.



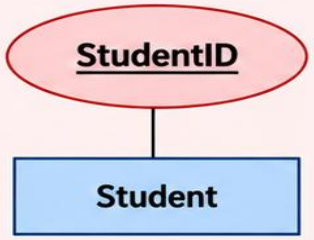


Entity–Relationship (ER) Model

Types of Attributes in ER Model

1 Key Attribute

- Uniquely identifies an entity instance.
- Represented by an underlined attribute.

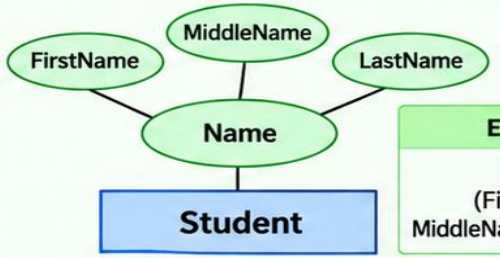


Example

StudentID

2 Composite Attribute

- Can be divided into smaller meaningful attributes.
- Represented by multiple ovals.

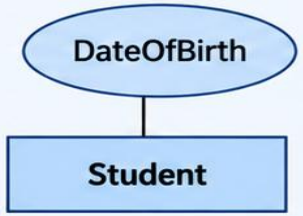


Example

Name (FirstName, MiddleName, LastName)

3 Single-valued Attribute

- Has only one value for each entity instance at a given time.
- Represented by a single oval.

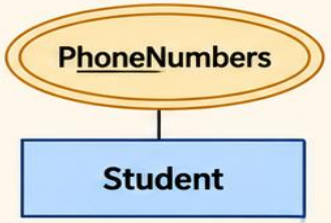


Example

DateOfBirth (e.g., 15-08-2000)

4 Multi-valued Attribute

- Can have multiple values for a single entity instance.
- Represented by a double oval.

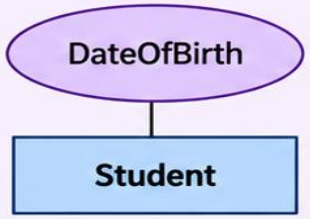


Example

PhoneNumbers (e.g., 9876543210, 9123456780)

5 Stored Attribute

- Value is stored in the database.
- Represented by a single oval (normal).

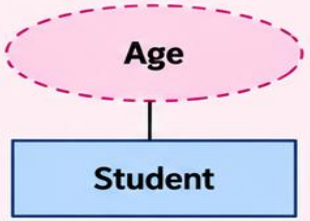


Example

DateOfBirth (stored value: 15-08-2000)

6 Derived Attribute

- Value is calculated from another attribute.
- Represented by a dashed oval.



Example

Age (derived from DateOfBirth)

Relationship in an ER Model

A relationship is an association between two or more entities in a database.

- It describes how entities are connected.
- In a traditional Chen ER diagram, a relationship is represented by a diamond (◇).
- A relationship can have attributes of its own.
- Relationships can be classified by cardinality, such as 1:1, 1:N, N:1, and M:N.

Example: Employee works for an Organization



•Explanation:

- Employee is an entity.

- Organization is an entity.

- Works For is the relationship between them.

- An employee works for an organization.

Another example

Entity 1	Relationship	Entity 2
Student	Enrolls in	Course
Employee	Works for	Organization
Department	Employs	Employee
Customer	Places	Order

Relationship in an ER Model

Types of relationships

Relationship	Meaning	Example
One-to-One (1:1)	One instance relates to one instance	Person — Passport
One-to-Many (1:N)	One instance relates to many instances	Department — Employees
Many-to-One (N:1)	Many instances relate to one instance	Employees — Department
Many-to-Many (M:N)	Many instances relate to many instances	Students — Courses

Choose an RDBMS when:

- Data follows a well-defined structure.
- Records relate to other records.
- Referential integrity is important.
- Transactions must maintain consistency.
- Reports require joins, filtering, and aggregation.

Typical applications:

- Banking and financial transactions
- College ERP and student management
- Hospital management
- Inventory and billing systems
- Payroll and employee management
- E-commerce orders and payments